DAFFODIL INTERNATIONAL UNIVERSITY
Dept. of Information and Communication Engineering

DIGITAL SIGNAL PROCESSING

A Comprehensive Manual for BSc Engineering Students

From Sampling to Modern DSP Applications:
Signals & Systems · Z-Transform · Frequency Analysis · DFT/FFT · Filter Realization & Design

Professor Dr. Engr. A.K.M. Fazlul Haque
Author

About This Manual

This manual has been prepared as a complete instructional reference for a BSc-level Digital Signal Processing course, based on the classic textbook by Proakis and Manolakis. It covers foundational theory (discrete-time signals and systems, the z-transform, frequency analysis) through applied, exam-style problem solving in the DFT/FFT, filter realization structures, fixed-point implementation, and digital filter design. Extra depth and additional worked examples are given to Chapters 1, 2, 3, 4, and 7 — the chapters most heavily emphasized in typical BSc DSP courses — while the manual closes with a survey of real-world DSP applications relevant to Bangladeshi engineering students.

Pedagogical Approach

Every chapter follows the same structure: (1) conceptual theory explained in plain, simple English, (2) an annotated diagram illustrating the concept, (3) MULTIPLE fully worked numerical examples with step-by-step solutions, and (4) review questions for self-assessment or classroom use. Examples are deliberately chosen to connect abstract formulas to familiar, everyday technology, including mobile networks and digital services widely used across Bangladesh.

Author

Professor Dr. Engr. A.K.M. Fazlul Haque

Preface: How to Use This Manual

This manual is written for BSc-level students in Bangladesh studying Digital Signal Processing as part of an ICE, EEE, or CSE programme. It is based on the classic textbook by Proakis and Manolakis, "Digital Signal Processing: Principles, Algorithms, and Applications," with material re-explained in simple, direct English and supplemented with additional worked examples connecting the theory to technologies Bangladeshi students already use every day — mobile networks, MP3/audio, and biomedical signals. Extra depth and additional examples have been given to Chapters 1, 2, 3, 4, and 7, which form the theoretical backbone most heavily emphasized in typical BSc DSP courses.

Course Week	Suggested Chapters	Focus
1 - 2	Chapter 1	Signals, sampling, quantization, the Sampling Theorem
3 - 4	Chapter 2	Sequences, LTI systems, convolution, difference equations
5 - 6	Chapter 3	The z-transform, pole-zero analysis, stability
7 - 8	Chapter 4	DTFT, frequency response, ideal filters
9	Chapter 5	The DFT and circular convolution
10	Chapter 6	FFT algorithms and computational savings
11 - 12	Chapter 7	Filter realization structures, fixed/floating point, quantization effects
13	Chapter 8	FIR/IIR filter design overview
14 - 15	Chapter 9	Applications and course wrap-up

Every chapter follows the same structure: theory explained in plain English, an annotated diagram, MULTIPLE fully worked numerical examples, and review questions. Students preparing for examinations should pay particular attention to the worked examples in Chapters 1-4 and 7, which are the most heavily tested topics in typical BSc DSP courses and examinations.

Table of Contents

Chapter 1: Introduction to Digital Signal Processing.....5

Chapter 2: Discrete-Time Signals and Systems.....13

Chapter 3: The Z-Transform and Its Application to the Analysis of LTI Systems.....21

Chapter 4: Frequency Analysis of Signals and Systems.....27

Chapter 5: The Discrete Fourier Transform (DFT) — Properties and Applications.....32

Chapter 6: Efficient Computation of the DFT — FFT Algorithms.....36

Chapter 7: Implementation of Discrete-Time Systems.....39

Chapter 8: Design of Digital Filters (FIR and IIR) — An Overview.....45

Chapter 9: Applications of Digital Signal Processing.....48

Appendix A: Master Formula Sheet.....51

Appendix B: MATLAB / Python (NumPy) Quick Reference.....53

Appendix C: Practice Examination (25 Questions).....54

Glossary of Key Terms.....56

Index.....57

Chapter 1: Introduction to Digital Signal Processing

Digital Signal Processing (DSP) is the branch of engineering that deals with representing signals in digital (numeric) form and processing them using mathematical algorithms, usually implemented in software running on a computer, microcontroller, or a specialized chip called a Digital Signal Processor. Every time you make a mobile phone call over Grameenphone, Robi, or Banglalink, listen to an MP3 song, watch a Blu-ray movie, or use a Bluetooth headset, digital signal processing is working behind the scenes.

1.1 Signals, Systems, and Signal Processing

A signal is any physical quantity that varies with time, space, or another independent variable, and that carries information. A human voice, an ECG waveform recorded in a hospital, a stock price recorded every day, and a black-and-white photograph (which is a 2-D signal, varying with two spatial coordinates) are all examples of signals. A system is any physical device (or algorithm) that performs an operation on a signal, transforming it in some useful way — for example, an amplifier, a filter, or a noise-cancellation algorithm in an earphone.

Signal Processing is the general name for the collection of techniques used to modify, analyze, or extract information from signals. When this processing is performed on a digital computer, using sequences of numbers rather than a continuously varying physical voltage, we call it Digital Signal Processing.

1.2 Basic Elements of a Digital Signal Processing System

Most real-world signals — sound, temperature, an image captured by a camera sensor — are naturally analog (continuous in time and continuous in amplitude). To process such a signal digitally, we must first convert it into a stream of numbers.

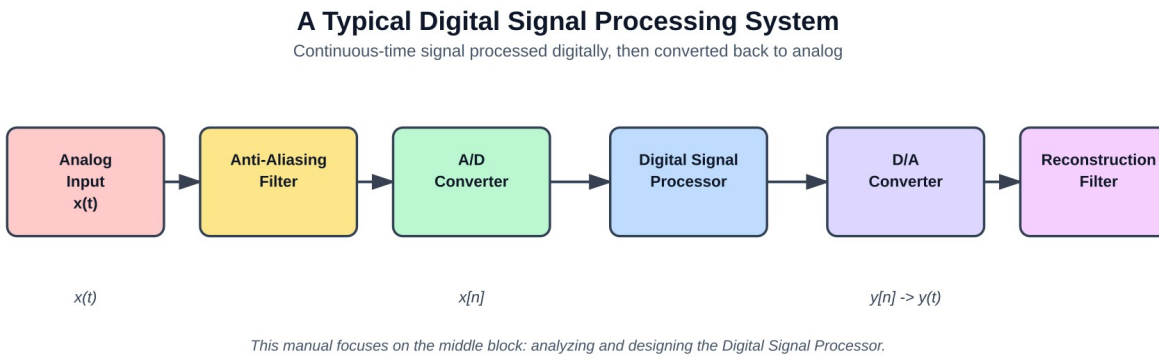


Figure 1.1 — A typical digital signal processing system: an analog signal is filtered, sampled, converted to digital form, processed, and (if needed) converted back to analog.

Block	Purpose
Anti-Aliasing Filter	An analog lowpass filter that removes frequency components above half the sampling rate, before sampling — this prevents a distortion called aliasing (Section 1.4.2)
A/D Converter (ADC)	Converts the filtered analog signal into a sequence of numbers (samples), each represented with a finite number of bits
Digital Signal Processor	A general-purpose computer, microcontroller, or specialized DSP chip that runs the actual processing algorithm (filtering, compression, recognition, etc.) on the sample sequence
D/A Converter (DAC)	Converts the processed digital sample sequence back into an analog voltage, when an analog output is required
Reconstruction Filter	A final analog smoothing (lowpass) filter that removes the "staircase" artifacts introduced by the D/A conversion process

1.3 Advantages of Digital over Analog Signal Processing

DSP has almost entirely replaced analog signal processing in modern electronics, for several compelling reasons that are worth understanding clearly.

- Precision and Reproducibility: a digital system built from the same design always behaves identically, whereas analog components (resistors, capacitors) vary slightly between units due to manufacturing tolerance, temperature, and aging.
- Flexibility: changing a digital filter's behaviour is often just a software update (think of a mobile phone firmware update improving call quality); changing an analog filter requires redesigning physical circuitry.

- Storage: digital signals (numbers) can be stored perfectly on a hard disk, USB drive, or cloud server, with no degradation over time, unlike analog magnetic tape which degrades with each playback.
- Advanced processing: complex operations like noise cancellation, speech recognition, and image compression are extremely difficult to implement with analog circuits, but are entirely practical as digital algorithms.
- Cost: thanks to the massive scale of the semiconductor industry, digital hardware (processors, memory) has become extremely inexpensive compared to precision analog components.

> **Note:** *The main disadvantage of DSP is the need for sampling and quantization (Section 1.4), which introduce two fundamental limitations: aliasing (if sampled too slowly) and quantization error (finite-precision rounding, studied in depth in Chapter 7). A well-designed DSP system manages both carefully.*

1.4 Classification of Signals

Signals can be classified along several independent dimensions. Understanding this classification vocabulary is essential, since it is used constantly throughout every later chapter.

Classification	Categories	Example
By number of channels	Single-channel (1-D) vs. Multichannel/Multidimensional	A mono audio signal (1-D) vs. a colour photograph, which has 3 channels — Red, Green, Blue — over a 2-D spatial grid
By time variable	Continuous-time vs. Discrete-time	A voltage waveform on an oscilloscope (continuous-time) vs. daily closing stock prices (naturally discrete-time)
By amplitude values	Continuous-valued (analog) vs. Discrete-valued (digital)	The exact voltage from a microphone (continuous-valued) vs. an 8-bit sample restricted to one of 256 possible levels (discrete-valued)
By predictability	Deterministic vs. Random (stochastic)	A pure sine wave generated by a signal generator (deterministic) vs. thermal noise in an electronic circuit (random)

A signal that is BOTH discrete-time AND discrete-valued is called a digital signal — this is exactly the kind of signal a computer or DSP chip works with internally.

1.5 The Concept of Frequency — Continuous-Time vs. Discrete-Time

Frequency is one of the most fundamental ideas in all of signal processing, but it behaves subtly differently for continuous-time and discrete-time signals — a distinction that trips up many students and deserves careful attention.

1.5.1 Continuous-Time Sinusoidal Signals

A continuous-time sinusoid is written as $x(t) = A\cos(\Omega t + \theta)$, where A is the amplitude, Ω is the frequency in radians/second ($\Omega = 2\pi F$, with F in Hertz), and θ is the phase. A key property: continuous-time sinusoids of DIFFERENT frequencies are always DIFFERENT signals, and increasing the frequency Ω always produces a signal that oscillates faster, without any limit.

1.5.2 Discrete-Time Sinusoidal Signals

A discrete-time sinusoid is written as $x[n] = A\cos(\omega n + \theta)$, where n is an integer sample index and ω is the digital (normalized) frequency in radians/sample. Discrete-time sinusoids have two surprising properties that have NO continuous-time counterpart:

- Periodicity in frequency: a discrete-time sinusoid with frequency ω is IDENTICAL to one with frequency $(\omega + 2\pi)$, $(\omega + 4\pi)$, and so on. This means we only ever need to consider ω in the range $-\pi$ to π (or 0 to 2π) — frequencies "wrap around" every 2π, unlike continuous-time frequency, which is unbounded.
- A discrete-time sinusoid is periodic (as a sequence) only if $\omega/2\pi$ is a RATIONAL number — an important and easily overlooked subtlety, since a continuous-time sinusoid is ALWAYS periodic.

Example 1.1

Problem Statement: Determine whether the discrete-time sequence $x[n] = \cos(0.3\pi n)$ is periodic, and if so, find its period N.

Solution:

(1) A discrete-time sinusoid $\cos(\omega n)$ is periodic if $\omega/2\pi = k/N$ for some integers k and N (i.e., $\omega/2\pi$ must be a rational number).

(2) Here $\omega = 0.3\pi$, so $\omega/2\pi = 0.3\pi/2\pi = 0.15 = 3/20$.

(3) This is a rational number (3/20), so the sequence IS periodic. Taking the fraction in lowest terms, k=3 and N=20.

Final Answer: **The sequence is periodic with fundamental period N = 20 samples.**

Example 1.2

Problem Statement: Determine whether x[n] = cos(3n) (note: no π here) is periodic.

Solution:

(4) Here ω = 3 (radians/sample), so ω/2π = 3/2π ≈ 0.4775…

(5) Since π is irrational, 3/2π is an IRRATIONAL number — it cannot be written as a ratio of two integers.

Final Answer: **The sequence is NOT periodic, even though it looks like an ordinary cosine wave. This is a classic discrete-time-only phenomenon that has no continuous-time analogy.**

1.6 Analog-to-Digital Conversion

Converting an analog signal to digital form involves three distinct steps: sampling (converting continuous time to discrete time), quantization (converting continuous amplitude to a finite set of discrete levels), and coding (representing each quantized level as a binary number).

1.6.1 Sampling of Analog Signals

Sampling takes the value of a continuous-time signal x(t) at uniformly spaced time instants: x[n] = x(nT), where T is the sampling period, and Fs = 1/T is the sampling frequency (or sampling rate), in Hertz (samples per second).

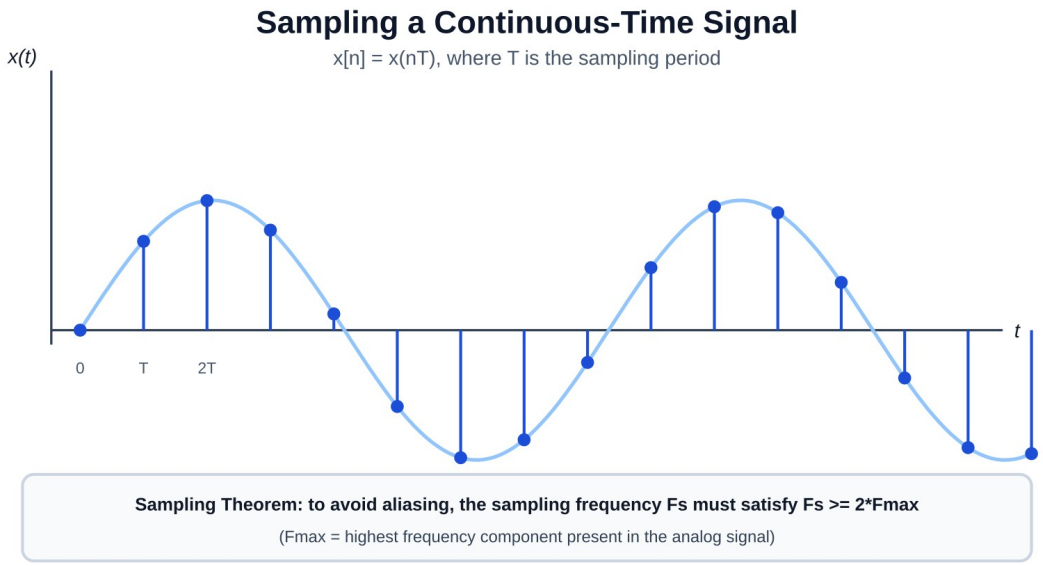


Figure 1.2 — Sampling a continuous-time signal at uniform intervals T to obtain a discrete-time sequence x[n].

1.6.2 The Sampling Theorem (Nyquist Theorem)

The single most important rule in all of DSP is the Sampling Theorem, first proven by Harry Nyquist and later formalized by Claude Shannon: a continuous-time signal with no frequency content above Fmax Hz can be COMPLETELY and PERFECTLY reconstructed from its samples, provided the sampling frequency satisfies:

```
Fs >= 2 * Fmax     (the "Nyquist rate" is exactly 2*Fmax)
```

> **Note:** *If Fs < 2*Fmax, a distortion called ALIASING occurs: high-frequency components in the original signal masquerade as (are indistinguishable from) lower-frequency components in the sampled sequence, and the original signal can never be recovered correctly, no matter how sophisticated the reconstruction process. This is why every ADC circuit is preceded by an anti-aliasing (lowpass) filter (Section 1.2).*

Example 1.3

Problem Statement: A Bangladeshi radio station broadcasts audio containing frequencies up to 15 kHz. What is the minimum sampling rate required to digitize this audio without aliasing?

Solution:

(6) Fmax = 15 kHz.

(7) By the Sampling Theorem, Fs >= 2 * Fmax = 2 * 15 kHz = 30 kHz.

Final Answer: **Minimum sampling rate = 30 kHz. (This is very close to why the standard audio CD sampling rate is 44.1 kHz — comfortably above the Nyquist rate for the full 20 kHz range of human hearing, allowing practical, non-ideal anti-aliasing filters some transition room.)**

Example 1.4

Problem Statement: A signal x(t) = 5cos(2π(1000)t) is sampled at Fs = 1500 Hz. Determine the frequency of the resulting aliased discrete-time signal.

Solution:

(8) The Nyquist rate here would need to be Fs >= 2(1000) = 2000 Hz, but we are only sampling at 1500 Hz — this violates the Sampling Theorem, so aliasing WILL occur.

(9) The apparent (aliased) frequency is found by reflecting the true frequency around multiples of Fs/2 = 750 Hz: Falias = |F - k*Fs| for the integer k that brings the result into the range [0, Fs/2].

(10) Falias = |1000 - 1*1500| = |-500| = 500 Hz.

Final Answer: **The reconstructed signal will incorrectly appear to have a frequency of 500 Hz, not the true 1000 Hz — a direct, practical demonstration of aliasing distortion.**

1.6.3 Quantization of Continuous-Amplitude Signals

After sampling, each sample's amplitude must be rounded to one of a finite number of discrete levels — this is quantization, and the rounding error introduced is called quantization error or quantization noise. If b bits are used per sample, there are 2^b possible quantization levels.

Example 1.5

Problem Statement: A signal in the range -5V to +5V is quantized using an 8-bit ADC. Find the quantization step size (resolution).

Solution:

(11) Total range = 5 - (-5) = 10V.

(12) Number of levels = 2^8 = 256.

(13) Step size (resolution) = Range / Number of levels = 10V / 256 = 0.03906 V ≈ 39.06 mV.

Final Answer: **Quantization step size ≈ 39.06 mV — the largest possible quantization error is half of this, about ±19.53 mV.**

1.7 Chapter Summary

- DSP processes signals as sequences of numbers, using algorithms running on a computer or DSP chip.
- A complete DSP system involves an anti-aliasing filter, an A/D converter, the digital processor itself, a D/A converter, and a reconstruction filter.
- Signals are classified by number of channels, time variable, amplitude values, and predictability.
- Discrete-time frequency ω is periodic with period 2π and, unlike continuous-time frequency, a discrete-time sinusoid is periodic only if ω/2π is rational.
- The Sampling Theorem requires Fs >= 2*Fmax to avoid aliasing — the single most important rule in DSP.
- Quantization introduces rounding error whose step size depends on the number of bits used per sample.

1.8 Review Questions

1. Explain, in your own words, why a digital voice call over a mobile network needs both an ADC (at the transmitting phone) and a DAC (at the receiving phone).
2. Is the sequence x[n] = sin(0.5n) periodic? Justify your answer using the rational/irrational frequency test.
3. A signal contains frequencies up to 8 kHz. What is the Nyquist rate, and what would happen if it were sampled at only 12 kHz?
4. A signal in the range 0V to 10V is quantized with a 10-bit ADC. Find the quantization step size.
5. List three advantages of digital over analog signal processing, with a Bangladeshi real-world example for each.

Chapter 2: Discrete-Time Signals and Systems

This chapter builds the mathematical toolkit for describing and analyzing discrete-time signals and the systems that process them. Nearly every later chapter — the z-transform, frequency analysis, filter design — is built directly on the ideas developed here, so it is worth mastering thoroughly.

2.1 Discrete-Time Signals: Representation

A discrete-time signal is a sequence of numbers, written x[n], where n is an integer (the sample index), NOT physical time in seconds. If the sequence came from sampling a physical signal x(t), then x[n] = x(nT), but a discrete-time sequence can also be a naturally discrete quantity, such as daily rainfall in Dhaka, with no underlying continuous-time signal at all.

2.1.1 Elementary Discrete-Time Signals



Figure 2.1 — The three most important elementary sequences: the unit impulse, the unit step, and the real exponential sequence.

Sequence	Definition	Role
Unit Impulse, δ[n]	1 if n=0, and 0 for all other n	The discrete-time "building block" — any sequence can be written as a weighted sum of shifted impulses; this is the basis of convolution (Section 2.3)
Unit Step, u[n]	1 for n>=0, and 0 for n<0	Used to represent signals that "switch on" at n=0, and to build other useful signals

Sequence	Definition	Role
Real Exponential, a^n u[n]	a raised to the power n, for n>=0 (zero otherwise)	Models decaying (|a|<1) or growing (|a|>1) behaviour; the natural response of many real systems (e.g., an RC circuit discharging) looks like this

2.1.2 Classification of Discrete-Time Signals

- Energy vs. Power Signals: a signal has finite ENERGY if the sum of |x[n]|^2 over all n is finite (e.g., a pulse that eventually dies out); a signal has finite average POWER if this sum, averaged over an increasingly large window, converges to a finite nonzero value (e.g., a pure, everlasting sinusoid).
- Periodic vs. Aperiodic: x[n] is periodic with period N if x[n+N] = x[n] for all n; otherwise it is aperiodic.
- Symmetric (Even) vs. Antisymmetric (Odd): x[n] is even if x[-n] = x[n], and odd if x[-n] = -x[n]. Any signal can be decomposed into a sum of an even part and an odd part.

2.1.3 Simple Manipulations of Discrete-Time Signals

Operation	Notation	Effect
Shifting	x[n - k]	Delays the sequence by k samples if k>0 (shifts it to the right); advances it if k<0
Folding (reflection)	x[-n]	Reverses (flips) the sequence in time about n=0
Scaling (amplitude)	a * x[n]	Multiplies every sample by a constant a
Addition/ Multiplication	x1[n] + x2[n], x1[n]*x2[n]	Sample-by-sample (pointwise) addition or multiplication of two sequences

Example 2.1

Problem Statement: Given x[n] = {2, 4, -1, 3} (defined for n = 0,1,2,3), sketch x[n-2] and x[-n].

Solution:

(14) x[n-2] shifts every value 2 samples to the RIGHT (delay): the new sequence is nonzero for n=2,3,4,5, with values {2,4,-1,3} at those positions respectively.

(15) x[-n] reflects the sequence about n=0: the original value at n=0 (which is 2) stays at n=0; the value at n=1 (4) moves to n=-1; the value at n=2 (-1) moves to n=-2; the value at n=3 (3) moves to n=-3.

Final Answer: x[n-2] is nonzero at n=2,3,4,5 with values 2,4,-1,3. x[-n] is nonzero at n=0,-1,-2,-3 with values 2,4,-1,3.

2.2 Discrete-Time Systems

A discrete-time system is any process (or device) that transforms an input sequence x[n] into an output sequence y[n], written y[n] = T{x[n]}. Systems are classified by several key properties, each of which determines whether powerful mathematical tools (like convolution and the z-transform) can be applied.

2.2.1 Classification of Discrete-Time Systems

Property	Definition	Why It Matters
Static (Memoryless) vs. Dynamic	Static: y[n] depends ONLY on x[n] at the same n. Dynamic: y[n] depends on past and/or future values of x[n] too	Dynamic systems need memory (storage of past values) to implement
Linear vs. Nonlinear	Linear if it satisfies BOTH additivity [T{x1+x2}=T{x1}+T{x2}] and homogeneity [T{a*x}=a*T{x}], jointly called superposition	Only linear systems can be analyzed with convolution, the z-transform, and frequency response — the tools of Chapters 2-4
Time-Invariant vs. Time-Varying	Time-invariant if a shift in the input causes an identical shift in the output, with no other change	Combined with linearity, gives us LTI systems — the central object of study in this entire manual
Causal vs. Noncausal	Causal if y[n] depends only on present and PAST inputs (never future inputs)	Every real-time physical system must be causal; noncausal systems can only be implemented with a processing delay, or off-line on stored data
Stable vs. Unstable	Stable (BIBO — Bounded Input, Bounded Output) if every bounded input produces a bounded output	An unstable system's output can grow without limit — clearly undesirable and often destructive in a real device

Example 2.2

Problem Statement: Determine whether the system y[n] = 2x[n] + 3 is linear.

Solution:

(16) Test homogeneity: apply input a*x[n]. Output = 2(a*x[n]) + 3 = 2a*x[n] + 3.

(17) Compare with a * (output for x[n]) = a * (2x[n] + 3) = 2a*x[n] + 3a.

(18) These are NOT equal in general (2a*x[n]+3 vs. 2a*x[n]+3a), because of the added constant "+3".

Final Answer: The system is NOT linear (it fails the homogeneity test). This is a common trap: systems with an added constant term always fail linearity, even though they look simple and "linear-ish" in everyday language.

Example 2.3

Problem Statement: Determine whether the system y[n] = x[n] * x[n-1] (product of current and previous sample) is time-invariant.

Solution:

(19) Apply a shifted input x[n-k]. The system produces: y1[n] = x[n-k] * x[n-k-1].

(20) Now shift the ORIGINAL output y[n] by k: y[n-k] = x[n-k] * x[(n-k)-1] = x[n-k] * x[n-k-1].

(21) y1[n] and y[n-k] are identical expressions.

Final Answer: The system IS time-invariant. (Note: this system is time-invariant but NOT linear, since it involves a product of the input with itself — a useful reminder that linearity and time-invariance are independent properties.)

2.3 Analysis of Discrete-Time Linear Time-Invariant (LTI) Systems

LTI systems are the single most important class of systems in this entire subject, because they can be COMPLETELY characterized by just one sequence: the impulse response, h[n] — the system's output when the input is a single unit impulse δ[n]. Once h[n] is known, the output for ANY input can be computed using convolution.

2.3.1 The Convolution Sum

Because any input x[n] can be written as a weighted sum of shifted impulses, and because the system is linear and time-invariant, the output of an LTI system to ANY input x[n] is given by the convolution sum:

```
y[n] = x[n] * h[n] = Σ x[k] * h[n-k]     (sum over all integers k,
from -infinity to +infinity)
```

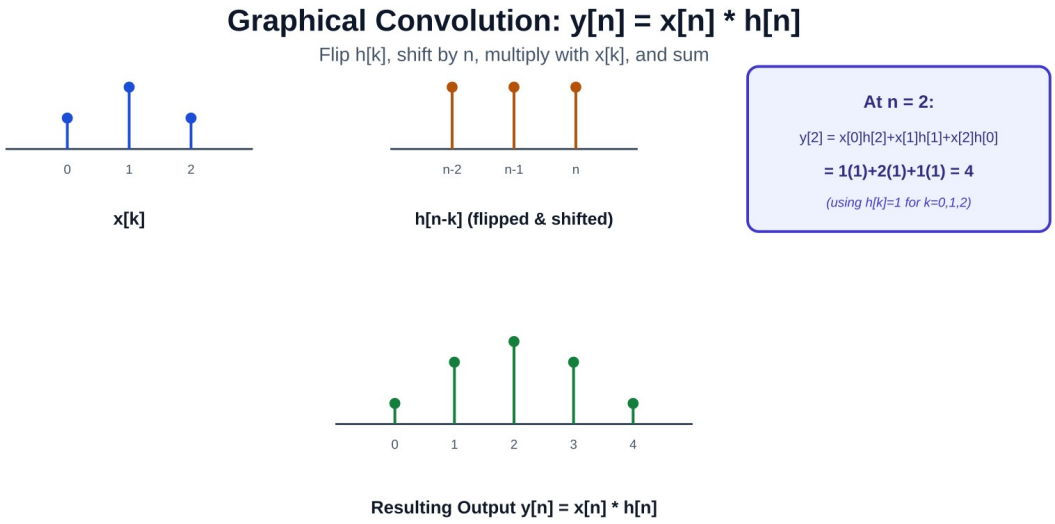


Figure 2.2 — Graphical convolution: flip h[k] to get h[-k], shift by n, multiply pointwise with x[k], and sum all the products to get y[n].

Example 2.4

Problem Statement: A system has impulse response h[n] = {1, 1, 1} for n=0,1,2. Find the output y[n] for input x[n] = {1, 2, 1} for n=0,1,2, using the convolution sum.

Solution:

(22) The output length will be Nx + Nh - 1 = 3 + 3 - 1 = 5 samples, for n=0 to 4.

(23) y[0] = x[0]h[0] = (1)(1) = 1

(24) y[1] = x[0]h[1] + x[1]h[0] = (1)(1) + (2)(1) = 3

(25) y[2] = x[0]h[2] + x[1]h[1] + x[2]h[0] = (1)(1) + (2)(1) + (1)(1) = 4

(26) y[3] = x[1]h[2] + x[2]h[1] = (2)(1) + (1)(1) = 3

(27) y[4] = x[2]h[2] = (1)(1) = 1

Final Answer: y[n] = {1, 3, 4, 3, 1} for n = 0,1,2,3,4. (This exact example is illustrated graphically in Figure 2.2.)

2.3.2 Properties of Convolution

Property	Statement
Commutative	x[n] * h[n] = h[n] * x[n] — the roles of input and impulse response can be swapped

Property	Statement
Associative	(x[n] * h1[n]) * h2[n] = x[n] * (h1[n] * h2[n]) — two systems in CASCADE (series) can be combined into one, with impulse response h1[n]*h2[n]
Distributive	x[n] * (h1[n] + h2[n]) = x[n]*h1[n] + x[n]*h2[n] — two systems in PARALLEL combine by simply ADDING their impulse responses

2.3.3 Causality and Stability of LTI Systems

For an LTI system, the general properties from Section 2.2.1 translate into simple, checkable conditions directly on the impulse response h[n]:

- An LTI system is CAUSAL if and only if h[n] = 0 for all n < 0 (the impulse response starts at or after time zero).
- An LTI system is STABLE (BIBO) if and only if the sum of |h[n]| over all n is FINITE — i.e., h[n] is absolutely summable.

Example 2.5

Problem Statement: A system has impulse response h[n] = (0.5)^n u[n]. Determine whether the system is causal and whether it is stable.

Solution:

(28) Causality: h[n] = 0 for n<0 (because of the u[n] factor), so the system IS causal.

(29) Stability: check if Σ|h[n]| is finite. Σ (0.5)^n for n=0 to infinity is a geometric series with ratio 0.5, which converges to 1/(1-0.5) = 2 — a finite value.

Final Answer: The system is both CAUSAL and STABLE.

Example 2.6

Problem Statement: A system has impulse response h[n] = (2)^n u[n]. Determine whether the system is stable.

Solution:

(30) Check Σ|h[n]| = Σ (2)^n for n=0 to infinity. Since the ratio (2) is greater than 1, this geometric series DIVERGES (grows without bound).

Final Answer: The system is causal (h[n]=0 for n<0) but UNSTABLE, since its impulse response is not absolutely summable — a bounded input could produce an unbounded output.

2.4 Discrete-Time Systems Described by Difference Equations

Many practical LTI systems (especially those implemented in real hardware or software) are described not by an explicit impulse response, but by a linear constant-coefficient difference equation:

```
y[n] = -Σ ak*y[n-k] + Σ bk*x[n-k]     (feedback terms ak,
feedforward terms bk)
```

Systems whose output depends on PAST OUTPUT values (feedback, the ak terms) are called recursive (or IIR — Infinite Impulse Response, studied further in Chapter 7); systems with no feedback (only bk terms) are called nonrecursive (or FIR — Finite Impulse Response).

Example 2.7

Problem Statement: A system is described by y[n] = 0.5 y[n-1] + x[n], with initial condition y[-1]=0. Find y[0], y[1], y[2] for input x[n] = u[n] (unit step).

Solution:

(31) y[0] = 0.5*y[-1] + x[0] = 0.5(0) + 1 = 1

(32) y[1] = 0.5*y[0] + x[1] = 0.5(1) + 1 = 1.5

(33) y[2] = 0.5*y[1] + x[2] = 0.5(1.5) + 1 = 1.75

Final Answer: y[0]=1, y[1]=1.5, y[2]=1.75 — notice the output is gradually approaching a steady-state value of 2 as n increases, a typical first-order recursive-system response.

2.5 Correlation of Discrete-Time Signals

Correlation measures the SIMILARITY between two signals (crosscorrelation) or between a signal and a time-shifted version of itself (autocorrelation). Correlation is used extensively in radar/sonar (detecting a known pulse echo), GPS signal acquisition, and pattern matching.

```
Crosscorrelation:  rxy[l] = Σ x[n] * y[n-l]     (l = lag)
```

> **Note:** *Correlation and convolution are computed with a very similar-looking formula, but they serve different purposes: convolution combines a system's impulse response with an input to find the output; correlation measures similarity/alignment between two signals. Mathematically, crosscorrelation is convolution of x[n] with the TIME-REVERSED y[n], i.e., rxy[l] = x[l] * y[-l].*

2.6 Chapter Summary

- Discrete-time signals are sequences x[n]; elementary sequences (impulse, step, exponential) are the building blocks for more complex signals.
- Systems are classified as static/dynamic, linear/nonlinear, time-invariant/time-varying, causal/noncausal, and stable/unstable.
- LTI systems are fully characterized by their impulse response h[n]; the output for any input is given by the convolution sum y[n]=x[n]*h[n].
- An LTI system is causal iff h[n]=0 for n<0, and stable iff h[n] is absolutely summable.
- Recursive (IIR) systems have feedback (depend on past outputs); nonrecursive (FIR) systems do not.
- Correlation measures similarity between signals and is closely related to, but distinct from, convolution.

2.7 Review Questions

6. Determine whether the system y[n] = n*x[n] is time-invariant.
7. A system has impulse response h[n] = (0.3)^n u[n]. Is it causal? Is it stable? Justify both answers.
8. Given x[n] = {1,1,1} (n=0,1,2) and h[n] = {1,2} (n=0,1), find y[n] = x[n]*h[n] using the convolution sum.
9. A system is described by y[n] = 0.25 y[n-1] + x[n], with y[-1]=0 and x[n]=u[n]. Find y[0], y[1], y[2].
10. Explain, in your own words, the practical difference between an FIR (nonrecursive) and an IIR (recursive) system.

Chapter 3: The Z-Transform and Its Application to the Analysis of LTI Systems

The z-transform converts a discrete-time sequence into a function of a complex variable z, transforming difficult convolution operations (Chapter 2) into simple multiplication, and transforming difference equations into ordinary algebraic equations. It plays exactly the same role in discrete-time systems that the Laplace transform plays in continuous-time systems, and it is the single most powerful analytical tool in this manual.

3.1 The Z-Transform

```
X(z) = Σ x[n] * z^-n     (sum over all integers n, from -infinity to +infinity)
```

Here z is a complex variable. This is called the bilateral (two-sided) z-transform. Because the infinite sum does not converge for every value of z, every z-transform must be reported together with its Region of Convergence (ROC) — the set of z values for which the sum converges to a finite value.

Example 3.1

Problem Statement: Find the z-transform of x[n] = a^n u[n] (a right-sided, causal exponential sequence).

Solution:

(34) X(z) = Σ (a^n u[n]) z^-n = Σ (a z^-1)^n, summed for n = 0 to infinity (since u[n]=0 for n<0).

(35) This is a geometric series with ratio (a z^-1). It converges (to 1/(1-a z^-1)) whenever |a z^-1| < 1, i.e., whenever |z| > |a|.

Final Answer: X(z) = 1/(1 - a z^-1) = z/(z-a), ROC: |z| > |a|. This is one of the most frequently used z-transform pairs in the entire subject — memorize it.

Example 3.2

Problem Statement: Find the z-transform of x[n] = -a^n u[-n-1] (a left-sided, anti-causal exponential).

Solution:

(36) X(z) = Σ (-a^n) z^-n for n = -infinity to -1. Substituting m=-n (m=1 to infinity): X(z) = -Σ a^-m z^m = -Σ (z/a)^m

(37) This geometric series converges when |z/a| < 1, i.e., |z| < |a|, giving X(z) = -((z/a)/(1-z/a)) = 1/(1-a z^-1).

Final Answer: X(z) = 1/(1 - a z^-1), ROC: |z| < |a|. Notice this has the EXACT SAME algebraic formula as Example 3.1, but a DIFFERENT ROC — this proves that the ROC is an essential, non-optional part of the z-transform; the formula alone does not uniquely identify the time-domain sequence.

3.2 Properties of the Z-Transform

Property	Time Domain	Z-Domain
Linearity	a*x1[n] + b*x2[n]	a*X1(z) + b*X2(z)
Time Shifting	x[n-k]	z^-k * X(z)
Time Reversal	x[-n]	X(1/z)
Convolution	x1[n] * x2[n]	X1(z) * X2(z) (ordinary multiplication!)
Differentiation in z	n * x[n]	-z * dX(z)/dz
Initial Value Theorem	x[0]	lim(z->infinity) X(z)

Note: *The Convolution Property is the whole reason the z-transform is so powerful: it converts the awkward convolution SUM of Chapter 2 into ordinary algebraic MULTIPLICATION. This single fact is what makes z-domain analysis of LTI systems dramatically easier than working directly in the time domain.*

3.3 The System Function H(z) and the Pole-Zero Plot

For an LTI system with impulse response h[n], the z-transform H(z) is called the System Function (or Transfer Function). By the convolution property, Y(z) = X(z) * H(z), so H(z) = Y(z)/X(z) — the system function is simply the ratio of output to input transforms.

For systems described by a linear constant-coefficient difference equation (Section 2.4), H(z) is always a RATIONAL function — a ratio of two polynomials in z (or z^-1). The roots of the numerator are called ZEROS, and the roots of the denominator are called POLES.

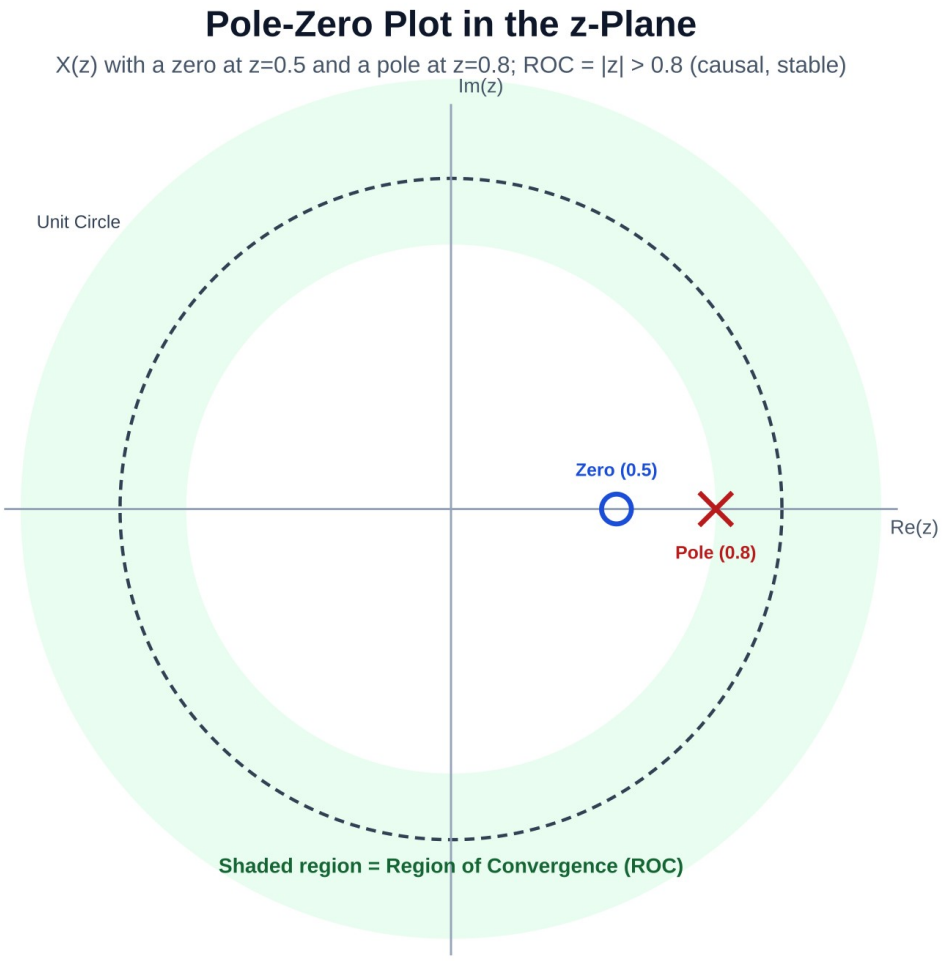


Figure 3.1 — A pole-zero plot in the z-plane. Zeros are marked with "o", poles with "x". The Region of Convergence never contains a pole.

3.4 Causality and Stability from the Pole-Zero Plot

One of the most elegant and practically useful results in this chapter is that we can determine BOTH causality and stability just by looking at the pole locations and the ROC, without ever computing the impulse response directly.

Condition	ROC Requirement
Causal system	ROC is the exterior of a circle, extending outward to infinity (ROC: $\|z\| > r$, for some radius r)
Stable system	ROC includes the unit circle $\|z\| = 1$
Causal AND Stable	ROC is $\|z\| > r$, with $r < 1$ — i.e., ALL poles must lie STRICTLY INSIDE the unit circle

Example 3.3

Problem Statement: A causal LTI system has H(z) = 1 / (1 - 0.8 z^-1). Determine whether the system is stable.

Solution:

(38) Rewrite: H(z) = z/(z-0.8). This has a single pole at z=0.8.

(39) Since the system is causal, the ROC is |z| > 0.8 (exterior of the circle through the outermost pole).

(40) Check if the ROC includes the unit circle: is 0.8 < 1? Yes.

Final Answer: The ROC (|z|>0.8) DOES include the unit circle, so the system IS stable. (This exact pole location, at z=0.8, is illustrated in Figure 3.1.)

Example 3.4

Problem Statement: A causal LTI system has H(z) = 1/(1 - 1.5 z^-1). Determine whether the system is stable.

Solution:

(41) This has a pole at z=1.5. Since the system is causal, ROC = |z| > 1.5.

(42) Is the unit circle (|z|=1) inside this ROC? No — 1 < 1.5, so the unit circle lies OUTSIDE (not within) the ROC |z|>1.5.

Final Answer: The system is UNSTABLE, because its single pole (z=1.5) lies OUTSIDE the unit circle. This matches the general rule: a causal system is stable if and only if all its poles lie strictly inside the unit circle.

3.5 Inversion of the Z-Transform

Given X(z) and its ROC, recovering the time-domain sequence x[n] is called inversion. The most practical technique for BSc-level problems, especially for rational functions, is Partial-Fraction Expansion, which breaks a complicated X(z) into a sum of simple terms matching known transform pairs (like Example 3.1).

Example 3.5

Problem Statement: Find the inverse z-transform of X(z) = z / [(z-0.5)(z-0.2)], given that the system is causal.

Solution:

(43) Write X(z)/z = 1/[(z-0.5)(z-0.2)] and expand by partial fractions: 1/[(z-0.5)(z-0.2)] = A/(z-0.5) + B/(z-0.2)

(44) Solving: A = 1/(0.5-0.2) = 1/0.3 = 3.333 ; B = 1/(0.2-0.5) = 1/(-0.3) = -3.333

(45) So X(z)/z = 3.333/(z-0.5) - 3.333/(z-0.2), giving X(z) = 3.333*z/(z-0.5) - 3.333*z/(z-0.2)

(46) Using the standard pair z/(z-a) <-> a^n u[n] (causal case, from Example 3.1):

Final Answer: x[n] = 3.333 (0.5)^n u[n] - 3.333 (0.2)^n u[n], or equivalently x[n] = 3.333[(0.5)^n - (0.2)^n] u[n].

3.6 Solving Difference Equations Using the Z-Transform

The z-transform converts a difference equation directly into an algebraic equation in z, which can be solved for Y(z) and then inverted — often far easier than solving the difference equation directly in the time domain, especially when initial conditions are involved (using the one-sided z-transform, Section 3.5 in the source text).

Example 3.6

Problem Statement: Solve y[n] - 0.5y[n-1] = x[n], with x[n] = u[n] and zero initial conditions, using the z-transform.

Solution:

(47) Take the z-transform of both sides: Y(z) - 0.5 z^-1 Y(z) = X(z)

(48) X(z) for a unit step is z/(z-1) (a standard pair, a special case of Example 3.1 with a=1).

(49) Y(z)[1 - 0.5z^-1] = z/(z-1) => Y(z) = z/[(z-1)(1-0.5z^-1)] = z^2/[(z-1)(z-0.5)]

(50) Expand Y(z)/z = z/[(z-1)(z-0.5)] by partial fractions: A/(z-1)+B/(z-0.5), with A=1/(1-0.5)=2, B=0.5/(0.5-1)=-1

(51) So Y(z) = 2z/(z-1) - z/(z-0.5)

Final Answer: y[n] = 2u[n] - (0.5)^n u[n] = [2 - (0.5)^n] u[n]. Check: y[0] = 2-1 = 1; y[1] = 2-0.5=1.5 — matching Example 2.7's result exactly, since that used the very same difference equation and input!

3.7 Chapter Summary

- The z-transform X(z) = Σ x[n]z^-n converts sequences into functions of a complex variable z; every X(z) must be paired with its ROC.
- The Convolution Property (convolution in time = multiplication in z) is the central reason the z-transform is so useful for LTI system analysis.
- The system function H(z) = Y(z)/X(z); its poles and zeros, together with the ROC, fully characterize the system.

- A causal system is stable if and only if all its poles lie strictly inside the unit circle.
- Partial-fraction expansion is the standard technique for inverting a rational X(z) back to x[n].
- The z-transform converts difference equations into algebraic equations, making it a powerful tool for solving them directly.

3.8 Review Questions

11. Find the z-transform (with ROC) of x[n] = (0.3)^n u[n].
12. A causal system has H(z) = 1/(1-0.9z^-1). Is it stable? Explain using the pole location.
13. Find the inverse z-transform of X(z) = z/[(z-0.4)(z-0.1)] (causal system).
14. State the Convolution Property of the z-transform and explain why it is so useful for analyzing LTI systems.
15. Solve y[n] - 0.25y[n-1] = x[n] for x[n]=u[n] with zero initial conditions, using the z-transform method.

Chapter 4: Frequency Analysis of Signals and Systems

While the z-transform (Chapter 3) is a general analytical tool, this chapter focuses on a special, extremely important case: evaluating the z-transform ON the unit circle (z = e^jw). This gives us the Discrete-Time Fourier Transform (DTFT), which tells us exactly how a signal or system behaves at every frequency — the foundation of filtering, spectral analysis, and communication engineering.

4.1 The Discrete-Time Fourier Transform (DTFT)

```
X(e^jw) = Σ x[n] * e^-jwn     (sum over all integers n)
```

The DTFT is simply the z-transform X(z) evaluated at z = e^jw, provided the unit circle lies within the ROC. X(e^jw) is a CONTINUOUS, PERIODIC function of w, with period 2π (a direct consequence of the frequency periodicity of discrete-time sinusoids discussed in Chapter 1).

Example 4.1

Problem Statement: Find the DTFT of x[n] = (0.5)^n u[n].

Solution:

(52) From Chapter 3, Example 3.1 (with a=0.5): X(z) = 1/(1-0.5z^-1), ROC |z|>0.5.

(53) Since the unit circle (|z|=1) lies inside this ROC (0.5 < 1), we can evaluate directly at z = e^jw.

Final Answer: X(e^jw) = 1/(1 - 0.5 e^-jw). This exact system is plotted in Figure 4.1, showing its lowpass-like magnitude response.

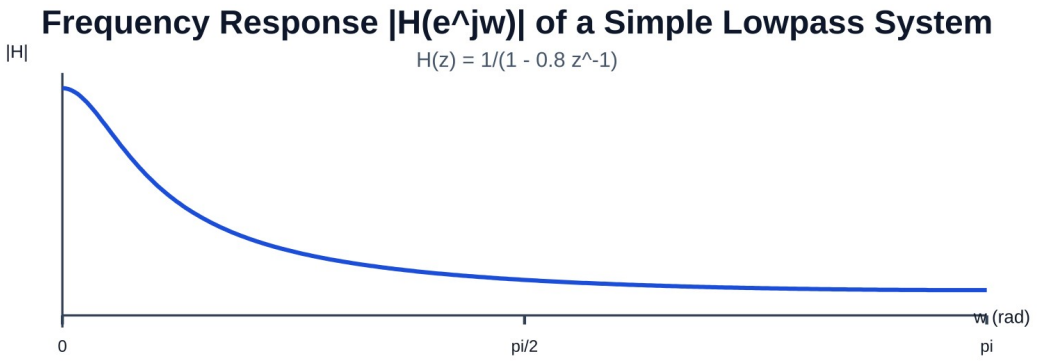


Figure 4.1 — Magnitude of the frequency response |H(e^jw)| for H(z)=1/(1-0.8z^-1), showing lowpass behaviour.

4.2 Magnitude and Phase Response

Since X(e^jw) is generally a complex number for each w, it is described using its MAGNITUDE |X(e^jw)| (how much of each frequency is present) and its PHASE (the angle of X(e^jw), representing the timing/delay of each frequency component). For an LTI system, |H(e^jw)| is called the magnitude response and the angle of H(e^jw) is the phase response.

Example 4.2

Problem Statement: For the system H(z) = 1/(1-0.8z^-1) (Example 3.3), find the magnitude response |H(e^jw)| at w=0 and w=π.

Solution:

(54) H(e^jw) = 1/(1 - 0.8 e^-jw)

(55) At w=0: e^-j0 = 1, so H(e^j0) = 1/(1-0.8) = 1/0.2 = 5. |H| = 5.

(56) At w=π: e^-jπ = -1, so H(e^jπ) = 1/(1-0.8(-1)) = 1/(1+0.8) = 1/1.8 = 0.556. |H| = 0.556.

Final Answer: |H| = 5 at DC (w=0), and only 0.556 at w=π (the highest discrete-time frequency) — confirming this system strongly favours low frequencies over high frequencies, i.e., it behaves as a LOWPASS filter, exactly as shown in Figure 4.1.

4.3 Properties of the Fourier Transform for Discrete-Time Signals

Property	Statement
Periodicity	X(e^jw) is always periodic in w with period 2π
Linearity	DTFT{a*x1[n]+b*x2[n]} = a*X1(e^jw) + b*X2(e^jw)
Time Shifting	DTFT{x[n-k]} = e^-jwk * X(e^jw) (a pure time delay only changes phase, not magnitude)
Convolution Theorem	DTFT{x[n]*h[n]} = X(e^jw) * H(e^jw) (ordinary multiplication of the two transforms)
Symmetry (real x[n])	If x[n] is real, |X(e^jw)| is EVEN in w, and the phase is ODD in w

4.4 Frequency-Domain Characteristics of LTI Systems

The Convolution Theorem tells us that if a sinusoid (or any signal) x[n] is passed through an LTI system with frequency response H(e^jw), the output's frequency content is simply the input's frequency content MULTIPLIED, frequency-by-frequency, by H(e^jw). This is the mathematical basis for describing what a filter "does" to a signal.

Example 4.3

Problem Statement: A sinusoid x[n] = cos(0.2πn) is applied to a system with H(e^jw) as in Example 4.2. Given |H(e^j0.2π)| = 3.1 and phase angle = -0.35 rad at this frequency, find the steady-state output y[n].

Solution:

(57) For an LTI system, a sinusoidal input at frequency w0 produces a sinusoidal output at the SAME frequency w0, scaled by |H(e^jw0)| and phase-shifted by the angle of H(e^jw0).

(58) y[n] = |H(e^jw0)| * cos(w0*n + phase) = 3.1 * cos(0.2πn - 0.35)

Final Answer: y[n] = 3.1 cos(0.2πn - 0.35). This is precisely why frequency response is so useful: once we know H(e^jw), the effect on ANY sinusoidal input can be found immediately, without solving any difference equation.

4.5 Linear Time-Invariant Systems as Frequency-Selective Filters

Building on Section 4.4, LTI systems are most often described by the SHAPE of their magnitude response, which determines which frequencies are passed through and which are blocked or attenuated — this is the basis of the term "filter".

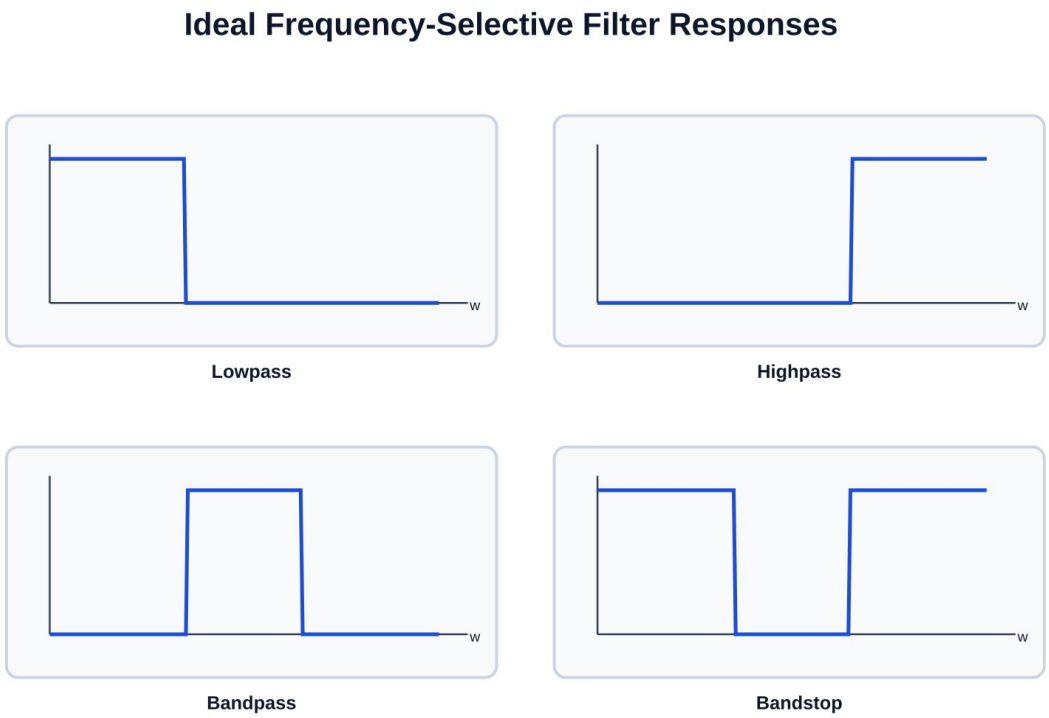


Figure 4.2 — Ideal magnitude responses of the four classic frequency-selective filter types.

Filter Type	Behaviour
Lowpass (LPF)	Passes frequencies below a cutoff wc, blocks frequencies above it — used to remove high-frequency noise, or as an anti-aliasing/reconstruction filter (Chapter 1)
Highpass (HPF)	Passes frequencies above wc, blocks frequencies below it — used to remove DC offset or slow drift from a signal
Bandpass (BPF)	Passes only a band of frequencies between two cutoffs — used to isolate one radio channel or one instrument's frequency range from a mixture
Bandstop (Notch)	Blocks only a narrow band of frequencies, passing everything else — commonly used to remove 50 Hz power-line hum from an audio or ECG signal (very relevant in Bangladesh, where mains frequency is 50 Hz)

4.6 Inverse Systems and Deconvolution

Given a system H(z), an INVERSE system Hi(z) is one that exactly undoes its effect: H(z) * Hi(z) = 1. Inverse systems are used in deconvolution — for example, removing the blurring effect of a camera lens from a photograph, or removing the distorting effect of a communication channel from a received signal (channel equalization in mobile phones).

> **Note:** *A stable, causal system has a stable, causal inverse only if it is MINIMUM-PHASE — meaning ALL of its poles AND zeros lie inside the unit circle. If any zero lies outside the unit circle, the exact inverse system would be either unstable or noncausal, and only an approximate inverse can be used in practice.*

4.7 Chapter Summary

- The DTFT X(e^jw) is the z-transform evaluated on the unit circle; it is continuous and periodic in w with period 2π.
- A signal/system's frequency-domain behaviour is described by magnitude response (gain per frequency) and phase response (delay per frequency).
- The Convolution Theorem converts time-domain convolution into simple multiplication in the frequency domain.
- LTI systems act as frequency-selective filters: lowpass, highpass, bandpass, and bandstop (notch) are the four classic ideal types.
- An inverse system undoes another system's effect; a stable causal inverse exists only for minimum-phase systems.

4.8 Review Questions

16. Find the DTFT of x[n] = (0.3)^n u[n].
17. For H(z) = 1/(1-0.5z^-1), find |H(e^jw)| at w=0 and w=π, and state whether this is a lowpass or highpass system.
18. State the Convolution Theorem for the DTFT and explain, in your own words, why it is useful for understanding filters.
19. Which type of ideal filter (LPF/HPF/BPF/Notch) would you use to remove 50 Hz power-line interference from a recorded ECG signal in a Bangladeshi hospital, and why?
20. What condition must a system satisfy for a stable, causal inverse system to exist?

Chapter 5: The Discrete Fourier Transform (DFT) — Properties and Applications

The DTFT (Chapter 4) is a CONTINUOUS function of frequency w — impossible to compute or store exactly on a digital computer using only finitely many numbers. The Discrete Fourier Transform (DFT) solves this practical problem by sampling the DTFT at a finite number of equally spaced frequencies, giving a transform that a computer can actually calculate.

5.1 Definition of the DFT

For a finite-length sequence x[n] of length N (n = 0 to N-1), the N-point DFT is defined as:

```
X[k] = Σ x[n] * e^-j(2π/N)kn     for k = 0, 1, ..., N-1     (sum
over n=0 to N-1)

Inverse DFT:  x[n] = (1/N) Σ X[k] * e^j(2π/N)kn
```

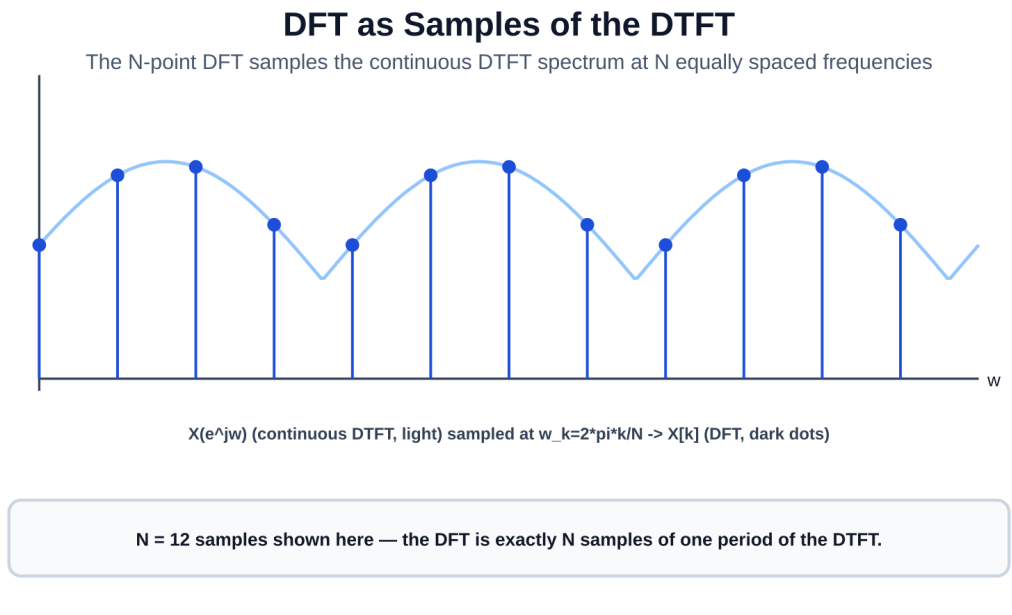


Figure 5.1 — The DFT is exactly N samples of one period of the continuous DTFT.

Example 5.1

Problem Statement: Find the 4-point DFT of x[n] = {1, 1, 0, 0} for n = 0,1,2,3.

Solution:

(59) X[k] = Σ x[n] e^-j(2π/4)kn = x[0] + x[1]e^-j(π/2)k, since x[2]=x[3]=0.

(60) X[0] = 1 + 1*e^0 = 1 + 1 = 2

(61) X[1] = 1 + 1*e^-j(π/2) = 1 + (-j) = 1 - j

(62) X[2] = 1 + 1*e^-jπ = 1 + (-1) = 0

(63) X[3] = 1 + 1*e^-j(3π/2) = 1 + j = 1 + j

Final Answer: X[k] = {2, 1-j, 0, 1+j} for k=0,1,2,3.

5.2 Properties of the DFT

Property	Statement
Periodicity	Both x[n] and X[k] are treated as periodic with period N when working with the DFT
Circular (not linear) Convolution	Multiplying two N-point DFTs and taking the inverse DFT gives CIRCULAR convolution of the two time-domain sequences, not ordinary (linear) convolution — an important and frequently tested distinction
Symmetry (real x[n])	If x[n] is real, X[N-k] = X*[k] (complex conjugate symmetry) — only about half the DFT values are independent
Parseval's Theorem	Σ|x[n]|^2 = (1/N) Σ|X[k]|^2 — energy is preserved (up to the 1/N scaling) between time and frequency domains

> **Note:** *Circular convolution vs. linear convolution is one of the most commonly confused topics for BSc students. To use the DFT to compute an ORDINARY (linear) convolution — as needed for filtering — both sequences must first be zero-padded to a length of at least (Nx + Nh - 1), so that the circular wrap-around does not corrupt the result.*

5.3 Linear Filtering Using the DFT

Because multiplying DFTs performs circular convolution, and filtering requires linear convolution, practical DFT-based filtering always uses zero-padding, following the length rule given in the note above (this same principle underlies the "Overlap-Add" and "Overlap-Save" methods used to filter very long signals in blocks).

Example 5.2

Problem Statement: x[n] has length 4 and h[n] has length 3. What is the minimum DFT length N required so that circular convolution (via the DFT) gives the SAME result as ordinary linear convolution?

Solution:

(64) Linear convolution length = Nx + Nh - 1 = 4 + 3 - 1 = 6.

(65) To avoid circular wrap-around corrupting the result, N must be at least this linear convolution length.

Final Answer: N >= 6 (commonly, the next convenient power of 2, N=8, would be chosen in practice for FFT efficiency — see Chapter 6).

5.4 Frequency Analysis Using the DFT

The DFT is the everyday, practical tool used to examine which frequencies are present in a real, recorded (finite-length) signal — for example, analyzing the frequency content of a recorded Rabindra Sangeet or folk-music sample, or diagnosing a mechanical vibration by inspecting its frequency spectrum. Each DFT index k corresponds to an actual physical frequency Fk = k * Fs / N Hz, where Fs is the sampling rate.

Example 5.3

Problem Statement: A signal sampled at Fs = 8000 Hz is analyzed with a 400-point DFT. What physical frequency does DFT bin k=50 correspond to?

Solution:

(66) Fk = k * Fs / N = 50 * 8000 / 400 = 400,000/400 = 1000 Hz.

Final Answer: DFT bin k=50 corresponds to 1000 Hz. This kind of calculation is essential every time a DFT/FFT spectrum is interpreted in a lab or real application.

5.5 Chapter Summary

- The DFT samples the continuous DTFT at N equally spaced frequencies, making frequency analysis computable on a digital machine.
- Multiplying DFTs corresponds to CIRCULAR convolution in time, not linear convolution — zero-padding is required to use the DFT for ordinary filtering.
- DFT bin k corresponds to physical frequency Fk = k*Fs/N Hz.
- Parseval's theorem relates time-domain energy to frequency-domain energy.

5.6 Review Questions

21. Find the 4-point DFT of x[n] = {1, 0, 1, 0}.
22. Explain the difference between circular and linear convolution, and why this matters for DFT-based filtering.

23. x[n] has length 5 and h[n] has length 5. What is the minimum DFT length for correct linear convolution via the DFT?

24. A signal sampled at 44100 Hz is analyzed with a 1024-point DFT. What frequency does bin k=100 correspond to?

Chapter 6: Efficient Computation of the DFT — FFT Algorithms

Computed directly from its definition, an N-point DFT requires on the order of N^2 complex multiplications — for a modest N=1024, that is over a million multiplications, repeated for every block of signal analyzed. The Fast Fourier Transform (FFT) is a family of algorithms that compute the EXACT SAME DFT result using only on the order of N*log2(N) multiplications — an enormous saving that is precisely what made real-time digital signal processing practical in the first place.

6.1 The Divide-and-Conquer Idea

The key insight behind the FFT (specifically, the radix-2 Decimation-In-Time algorithm) is to split an N-point DFT (with N a power of 2) into two smaller (N/2)-point DFTs — one of the even-indexed samples, one of the odd-indexed samples — and then combine their results using a simple computational unit called a butterfly. This splitting is applied recursively, all the way down to trivial 2-point DFTs.

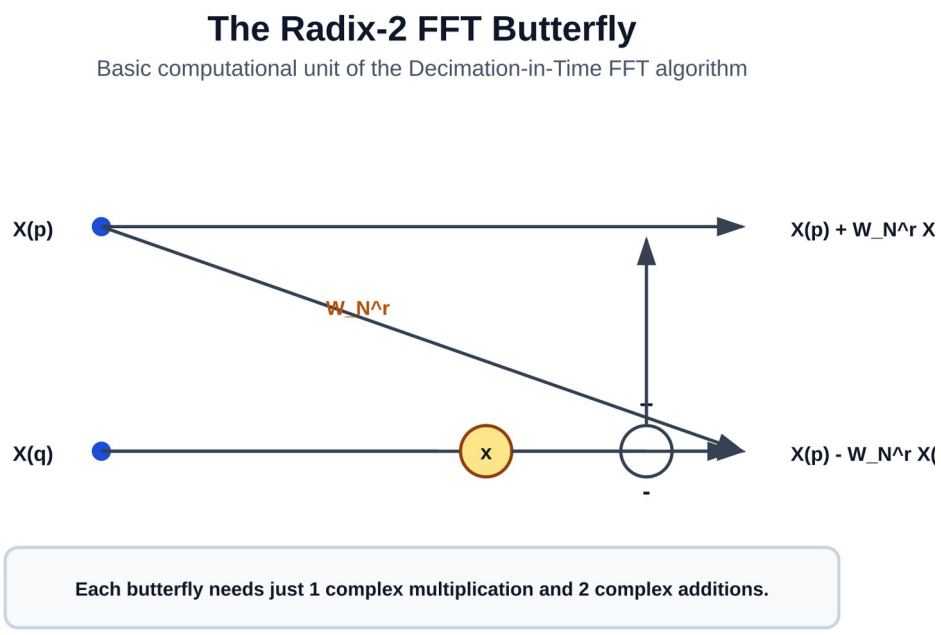


Figure 6.1 — The basic radix-2 FFT butterfly: one complex multiplication and two complex additions combine two inputs into two outputs.

6.2 Computational Savings: Direct DFT vs. FFT

N	Direct DFT: N^2 multiplications	FFT: (N/2)log2(N) multiplications	Speedup Factor
8	64	12	5.3x
64	4,096	192	21.3x
1024	1,048,576	5,120	204.8x
1,048,576 (2^20)	~1.1 x 10^12	~10,485,760	~100,000x

Example 6.1

Problem Statement: Calculate the number of complex multiplications needed for a direct 256-point DFT versus a radix-2 FFT.

Solution:

(67) Direct DFT: N^2 = 256^2 = 65,536 multiplications.

(68) FFT: (N/2)*log2(N) = (256/2)*log2(256) = 128 * 8 = 1,024 multiplications.

(69) Speedup factor = 65,536 / 1,024 = 64x.

Final Answer: The FFT needs only 1,024 multiplications versus 65,536 for the direct method — a 64-times speedup, which is exactly why every real-time spectrum analyzer, audio codec, and mobile-network baseband processor uses the FFT rather than the direct DFT formula.

6.3 Radix-2 FFT Requirements

The classic radix-2 Decimation-in-Time (and Decimation-in-Frequency) FFT algorithms require N to be an exact power of 2 (N = 2, 4, 8, 16, ...). If the actual signal length is not a power of 2, it is simply ZERO-PADDED up to the next convenient power of 2 before applying the FFT — this does not change the underlying frequency content being analyzed, only the frequency-domain sampling resolution (recall Section 5.4).

6.4 Applications of the FFT

- Fast, efficient linear filtering of long signals (via the Overlap-Add / Overlap-Save methods, built on the zero-padding principle of Section 5.3).
- Real-time spectrum analysis in test equipment, software-defined radios, and mobile network signal processing (e.g., OFDM, the modulation scheme underlying 4G/5G and Wi-Fi, is fundamentally built on the FFT/IFFT).

- Audio and image compression standards (MP3, JPEG use closely related transforms — MP3 uses a Modified DCT, itself derived from FFT-related mathematics).
- Fast computation of correlation (Chapter 2) for radar, sonar, and GPS signal acquisition.

6.5 Chapter Summary

- The FFT computes the exact same result as the direct DFT formula, but using far fewer operations: O(N log N) instead of O(N^2).
- The radix-2 FFT recursively splits the DFT into even- and odd-indexed sub-DFTs, combined using butterfly operations.
- Radix-2 FFT requires N to be a power of 2; non-power-of-2 lengths are zero-padded.
- The FFT underlies nearly all practical real-time frequency analysis and is a core building block of modern mobile communication standards (OFDM in 4G/5G/Wi-Fi).

6.6 Review Questions

25. Calculate the number of complex multiplications for a direct DFT versus a radix-2 FFT for N=512.
26. Why must N be a power of 2 for the classic radix-2 FFT algorithm, and what is done when the actual data length is not a power of 2?
27. Name two real-world applications (ideally with a Bangladeshi-relevant example) that rely on the FFT.
28. Explain, in your own words, the basic "divide-and-conquer" idea behind the FFT.

Chapter 7: Implementation of Discrete-Time Systems

Chapters 2-4 taught us how to ANALYZE a discrete-time system, given its difference equation or system function. This chapter addresses an equally important, very practical question: given a system function H(z), how do we actually BUILD it — as a hardware circuit, a DSP-chip program, or software — and what real-world limitations (finite-precision arithmetic) must an engineer account for?

7.1 Structures for the Realization of Discrete-Time Systems

A "structure" or "realization" is a specific arrangement of adders, multipliers, and unit-delay elements that implements a given difference equation. The SAME system function H(z) can be realized using several different, mathematically equivalent structures — a choice that matters enormously in practice, because different structures have different memory requirements, computational cost, and (critically, Section 7.6-7.7) different sensitivity to finite-precision arithmetic errors.

7.2 Structures for FIR Systems

An FIR (Finite Impulse Response) system has a difference equation with NO feedback: y[n] = Σ bk x[n-k]. The most direct structure is called the Direct Form.

Example 7.1

Problem Statement: Draw (describe) the Direct Form structure for the 3-tap FIR filter y[n] = 2x[n] + 3x[n-1] - x[n-2], and count the number of multipliers, adders, and delay elements required.

Solution:

(70) The input x[n] passes through a chain of 2 unit-delay elements, producing x[n], x[n-1], x[n-2] simultaneously available at each time step.

(71) Each tapped value is multiplied by its corresponding coefficient: 2, 3, and -1 respectively.

(72) The three products are summed to produce y[n].

Final Answer: 3 multipliers, 2 adders, and 2 delay elements are required — in general, an FIR filter with M+1 coefficients (taps) needs M+1 multipliers, M adders, and M delay elements.

7.3 Structures for IIR Systems

An IIR (Infinite Impulse Response) system has feedback (recall Chapter 2, Section 2.4): y[n] = -Σ ak y[n-k] + Σ bk x[n-k]. Two especially important, commonly examined structures are Direct Form I and Direct Form II.

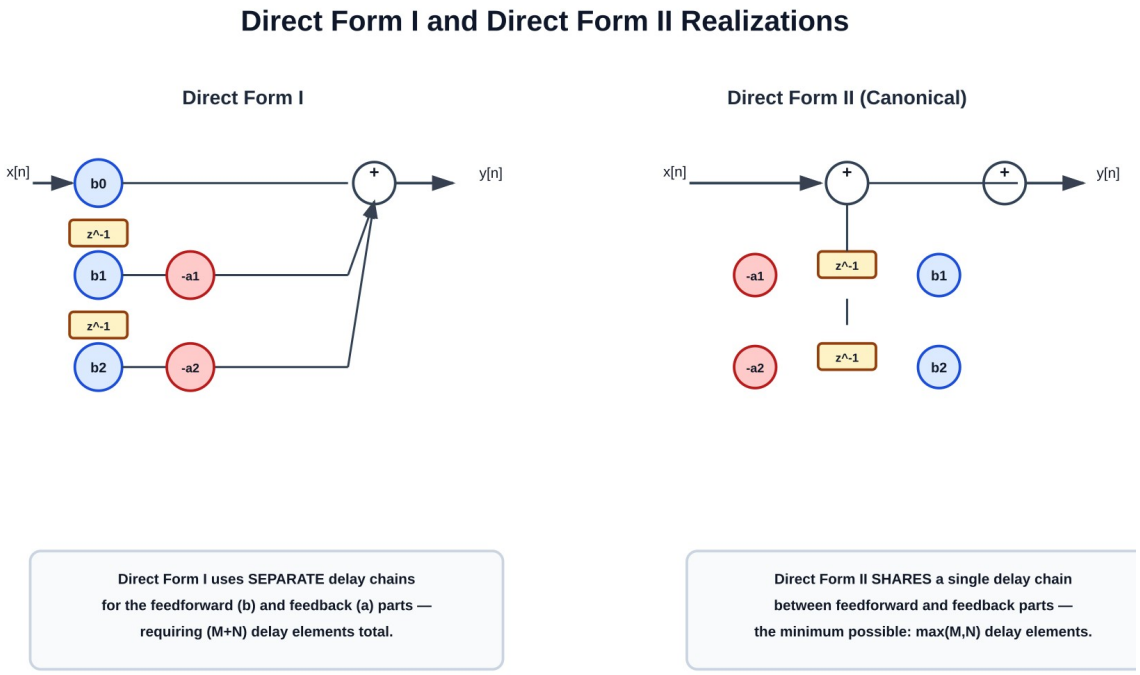


Figure 7.1 — Direct Form I (separate delay chains for feedforward and feedback) versus Direct Form II (a single, shared delay chain — the minimum possible).

Structure	Delay Elements Needed	Key Feature
Direct Form I	M + N (separate chains for b's and a's)	Simple, direct translation of the difference equation; not the most memory-efficient
Direct Form II (Canonical)	max(M, N) — the theoretical minimum possible	Shares ONE delay chain between the feedforward and feedback parts, halving memory in the common case M=N
Cascade Form	Same total as Direct Form II	H(z) factored into a product of small (usually 2nd-order) sections in series — much less sensitive to coefficient-quantization errors (Section 7.6)

Structure	Delay Elements Needed	Key Feature
Parallel Form	Same total as Direct Form II	H(z) expanded (via partial fractions) into a SUM of small sections computed in parallel — also more robust to quantization than a single high-order Direct Form

> **Note:** *Exam Tip: "Canonical" structure means using the MINIMUM possible number of delay elements — Direct Form II is canonical, Direct Form I generally is not (unless M=0 or N=0).*

7.4 State-Space System Analysis and Structures

State-space representation describes a system using a set of internal "state variables" and a system of FIRST-ORDER matrix difference equations: q[n+1] = A q[n] + B x[n], and y[n] = C q[n] + D x[n]. This representation is especially important for multi-input, multi-output (MIMO) systems, and is the standard language used in modern control systems and advanced filter implementation.

7.5 Representation of Numbers

Every real DSP implementation (a DSP chip, an FPGA, a microcontroller) stores numbers using a FINITE number of bits, which introduces two related but distinct practical issues covered in the rest of this chapter: how numbers are represented, and the errors that result from that finite representation.

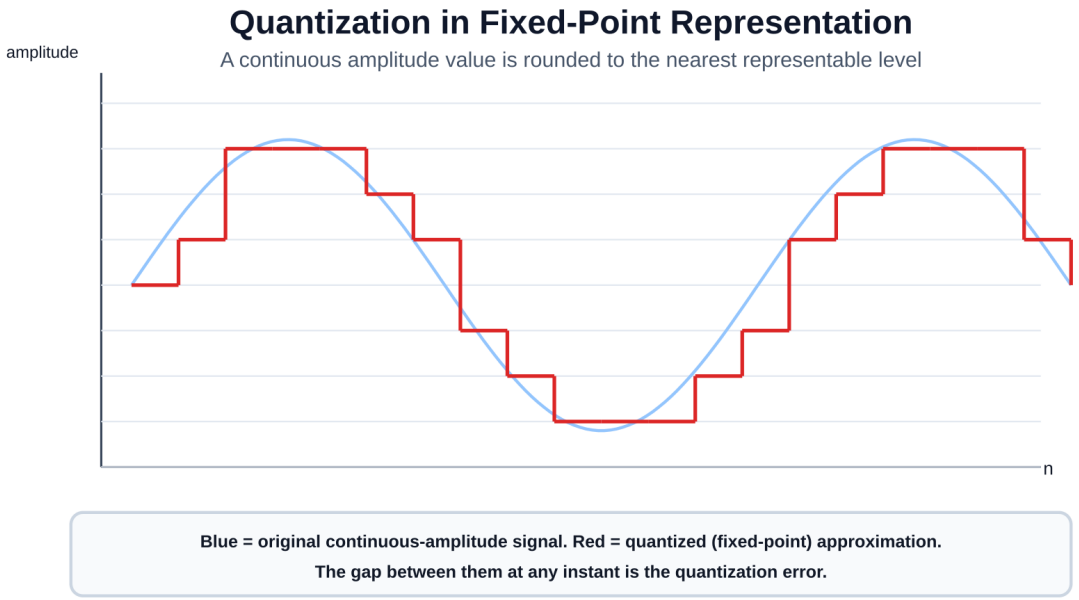


Figure 7.2 — Quantization: a continuous-amplitude value is rounded to the nearest representable fixed-point level.

Representation	Description
Fixed-Point	A number is represented as an integer with an assumed (fixed) binary point position — simple, fast, and inexpensive in hardware, but has a limited dynamic range and requires careful attention to overflow
Floating-Point	A number is represented as (mantissa) x 2^(exponent), similar to scientific notation — offers a huge dynamic range and simpler-to-manage precision, at the cost of more complex (traditionally slower and more power-hungry) hardware

Example 7.2

Problem Statement: A fixed-point system uses 12-bit two's-complement representation for a signal in the range -1 to +1 (a common DSP convention). Find the quantization step size and the maximum quantization error.

Solution:

(73) With 12 bits, there are 2^12 = 4096 representable levels across the range from -1 to just under +1 (total span = 2).

(74) Quantization step size, $\Delta = 2 / 4096 = 0.000488$ (approximately 4.88 x 10^-4).

(75) For rounding (rather than truncation), the maximum quantization error is half the step size: $\Delta/2 = 0.000244$.

Final Answer: **Step size Δ ≈ 4.88 x 10^-4 ; Maximum quantization error ≈ ±2.44 x 10^-4. Doubling the number of bits (e.g., going from 12 to 13 bits) always HALVES both of these values — one extra bit always doubles precision.**

7.6 Quantization of Filter Coefficients

When a filter's ideal, real-valued coefficients (a's and b's) are rounded to fit a finite-precision fixed-point representation, the ACTUAL poles and zeros of the implemented filter shift slightly away from their intended, ideally designed locations. For a HIGH-ORDER filter realized as a single Direct Form structure, this sensitivity can be severe enough to move a pole outside the unit circle — turning a designed-stable filter into an actually-UNSTABLE implementation. This is precisely why the Cascade and Parallel forms (Section 7.3), built from robust low-order (1st/2nd-order) sections, are strongly preferred for realizing high-order filters in fixed-point hardware.

7.7 Round-Off Effects in Digital Filters

Beyond coefficient quantization (a one-time design-stage effect), every ADDITION and MULTIPLICATION performed WHILE THE FILTER IS RUNNING also introduces a small rounding error at every single time step, because a fixed-point multiplier's product must itself be rounded back down to the system's fixed word length before the next operation.

Effect	Description
Limit-Cycle Oscillations	In recursive (IIR) systems, accumulated round-off error can cause a small, persistent, unwanted low-level oscillation in the output even when the true input is exactly zero — a phenomenon with no counterpart in ideal (infinite-precision) systems
Overflow	If an intermediate sum exceeds the maximum representable fixed-point value, the result "wraps around" catastrophically (in two's-complement arithmetic) unless SCALING (deliberately reducing signal levels at intermediate points) is used to keep all internal sums safely within range
Accumulated Round-off Noise	The statistical accumulation of many small per-step rounding errors, which can be analyzed and predicted using noise-power models, much like thermal noise analysis in analog circuits

> **Note:** *Practical engineering takeaway: this is exactly why real DSP chips and audio/communication systems either (a) use floating-point arithmetic where dynamic range and precision are critical, or (b) use fixed-point arithmetic together with careful scaling and low-order Cascade/Parallel structures, rather than a single naive high-order Direct Form I realization.*

7.8 Chapter Summary

- The same H(z) can be realized with different structures (Direct Form I/II, Cascade, Parallel, State-Space), each with different memory and robustness trade-offs.
- Direct Form II is "canonical" — it uses the minimum possible number of delay elements, max(M,N).
- Numbers in real DSP hardware are represented in fixed-point (fast, limited range) or floating-point (flexible, more complex) format.
- Quantizing filter coefficients shifts the actual poles/zeros away from their ideal design — a real risk of instability for high-order Direct Form filters.
- Round-off during filter operation causes limit-cycle oscillations, overflow risk, and accumulated round-off noise — all mitigated by using Cascade/Parallel structures and careful scaling.

7.9 Review Questions

29. How many multipliers, adders, and delay elements are needed for a 5-tap Direct Form FIR filter?
30. Explain why Direct Form II is called a "canonical" structure.
31. A fixed-point system uses 8-bit representation over the range -1 to +1. Find the quantization step size and maximum quantization error.
32. Why are Cascade and Parallel structures generally preferred over a single high-order Direct Form realization in fixed-point hardware?
33. What is a limit-cycle oscillation, and in which type of system (FIR or IIR) does it occur?

Chapter 8: Design of Digital Filters (FIR and IIR) — An Overview

Chapters 4 and 7 explained how to ANALYZE a filter's frequency response and how to IMPLEMENT a given filter, respectively. This chapter briefly surveys how filter COEFFICIENTS are actually designed in the first place, to meet a required specification (e.g., "lowpass, cutoff 3 kHz, at least 40 dB stopband attenuation").

8.1 Filter Design Specifications

A practical filter specification is never an ideal "brick wall" (Chapter 4, Figure 4.2); it always allows a transition band and specifies acceptable ripple/attenuation limits.

Parameter	Meaning
Passband ripple	The maximum allowed variation in gain within the passband (should be close to, but need not be exactly, 1)
Stopband attenuation	The minimum required attenuation (signal reduction) within the stopband, usually specified in dB
Transition band	The frequency range between the passband and stopband edges, where the response is allowed to roll off gradually

8.2 FIR Filter Design Methods

Method	Basic Idea
Windowing Method	Start from the IDEAL (infinite-length) impulse response, truncate it to a finite length, and multiply by a smooth "window" function (Hamming, Hanning, Blackman, etc.) to reduce the ripple (Gibbs phenomenon) caused by abrupt truncation
Frequency-Sampling Method	Specify the desired frequency response directly at a set of equally spaced frequencies, then compute the corresponding (inverse DFT-based) impulse response
Optimal (Parks-McClellan / Equiripple) Method	Uses an iterative computer algorithm to design a filter whose passband and stopband error is spread EQUALLY (equiripple) across the band — for a given filter length, this produces the best possible (minimum maximum-error) filter

> **Note:** *FIR filters are always stable (they have no feedback, hence no poles other than at the origin) and can easily be designed to have exactly LINEAR PHASE (by making the coefficients symmetric), which means every frequency component is delayed by the same amount of time — extremely important in applications like image and video processing where phase distortion is visually very noticeable.*

8.3 IIR Filter Design Methods

A very common and practically important approach to IIR filter design starts from decades of well-understood ANALOG filter design theory, and converts an analog design into an equivalent digital filter.

Analog Prototype	Characteristic
Butterworth	Maximally flat magnitude response in the passband (no ripple), but a relatively gradual roll-off
Chebyshev Type I	Equiripple in the passband, monotonic in the stopband — sharper roll-off than Butterworth for the same filter order
Chebyshev Type II	Monotonic passband, equiripple stopband
Elliptic (Cauer)	Equiripple in BOTH passband and stopband — the sharpest possible roll-off for a given filter order, at the cost of more phase distortion

The most common technique for converting an analog prototype into a digital IIR filter is the Bilinear Transformation, which maps the entire (infinite) analog frequency axis onto the (finite, 0 to π) digital frequency range in a one-to-one, stable-to-stable manner, avoiding the aliasing problems of simpler conversion methods.

8.4 FIR vs. IIR — Design Trade-offs

Aspect	FIR	IIR
Stability	Always stable	Can be unstable if poles move outside the unit circle
Phase response	Can be made exactly linear phase	Generally nonlinear phase
Filter order for a given specification	Usually needs a HIGHER order (more taps/computation) for a sharp response	Usually achieves a similar specification with a LOWER order (less computation)
Typical use case	Applications needing linear phase — image processing, high-quality audio	Applications where computational efficiency matters more than phase linearity

8.5 Chapter Summary

- Real filter specifications allow passband ripple, stopband attenuation, and a transition band — not an ideal brick wall.
- FIR filters are commonly designed using windowing, frequency sampling, or the optimal Parks-McClellan (equiripple) method; they are always stable and can have exactly linear phase.

- IIR filters are commonly designed from analog prototypes (Butterworth, Chebyshev, Elliptic) converted via the Bilinear Transformation; they need a lower order but are not guaranteed stable and generally have nonlinear phase.

8.6 Review Questions

34. Why are FIR filters always stable, while IIR filters are not?

35. Which analog filter prototype gives the sharpest roll-off for a given order, and what is the trade-off?

36. Explain why FIR filters can be designed with exactly linear phase, while typical IIR filters cannot.

37. Name one BSc-lab-relevant application where linear phase is important, and explain why.

Chapter 9: Applications of Digital Signal Processing

This closing chapter connects every earlier chapter to real, everyday DSP applications — many of which Bangladeshi BSc ICE students encounter directly in daily life — and points toward further study.

9.1 Speech and Audio Processing

Mobile voice calls (2G/3G/4G/5G) use speech codecs (like AMR, used across Bangladeshi mobile networks) that apply linear prediction — a direct application of the difference-equation and z-transform theory of Chapters 2-3 — to compress speech to a fraction of its original size before transmission. Music streaming and MP3/AAC files use FFT-based (Chapter 6) transform coding to remove psychoacoustically inaudible frequency content.

9.2 Digital Communication Systems

Modern mobile and Wi-Fi standards (4G LTE, 5G NR, 802.11 Wi-Fi) use OFDM (Orthogonal Frequency Division Multiplexing), which is built directly on the FFT/IFFT (Chapter 6) to simultaneously transmit data over many parallel narrowband subcarriers, dramatically improving robustness against multipath fading in real Bangladeshi urban and rural radio environments.

9.3 Biomedical Signal Processing

ECG (electrocardiogram) and EEG (electroencephalogram) signals recorded in Bangladeshi hospitals are digitally filtered (Chapters 4, 7-8) to remove 50 Hz power-line interference (using a notch filter) and baseline wander (using a highpass filter), and are analyzed using DFT/FFT-based spectral analysis (Chapters 5-6) to detect abnormal heart rhythms or brain activity.

9.4 Image and Video Processing

A digital image is a 2-D discrete signal; 2-D extensions of the DFT and FIR filtering (Chapters 5, 7-8) are used for blurring, sharpening, edge detection, and compression (JPEG uses a closely related transform, the Discrete Cosine Transform). Video calling apps widely used in Bangladesh (WhatsApp, Zoom, Google Meet) rely on real-time DSP for both audio and video compression.

9.5 Radar, Sonar, and GPS

Correlation (Chapter 2) is used to detect a known transmitted pulse buried in noise — the basic principle behind radar (weather radar used by the Bangladesh Meteorological Department) and GPS receivers (used in ride-sharing apps like Pathao and Uber for navigation), which correlate the received satellite signal against known reference codes to determine precise timing and position.

9.6 Where to Go From Here

Students who complete this manual are well prepared to study more advanced topics: Adaptive Filters (filters that automatically adjust their coefficients, used in echo cancellation for phone calls), Multirate Signal Processing (changing the sampling rate efficiently, used whenever audio must be converted between different standard rates, e.g., 44.1 kHz music mixed with 48 kHz video), and Statistical/Random Signal Processing (analyzing signals corrupted by genuinely random noise, extending the deterministic-signal theory of this manual).

9.7 Chapter Summary

- Speech/audio compression (mobile calls, MP3/AAC) relies directly on linear prediction and FFT-based transform coding.
- Modern wireless standards (4G/5G/Wi-Fi) use OFDM, built on the FFT/IFFT.
- Biomedical signal processing uses digital filtering and spectral analysis to clean and interpret ECG/EEG signals.
- Image/video processing extends 1-D DFT and FIR filtering concepts into two dimensions.
- Radar, sonar, and GPS rely on correlation to detect known signals within noise.

9.8 Review Questions

38. Name three DSP applications you personally use every day in Bangladesh, and identify which chapter's theory underlies each.

39. Why is the FFT specifically (rather than the direct DFT) essential for real-time applications like OFDM-based mobile networks?

40. Explain, in your own words, how correlation allows a GPS receiver to determine its distance from a satellite.

41. Which topic covered in this manual would you most like to study further, and why?

Appendix A: Master Formula Sheet

A.1 Sampling and Quantization (Chapter 1)

Quantity	Formula
Sampling	x[n] = x(nT), Fs = 1/T
Sampling Theorem	Fs >= 2 Fmax
Aliased frequency	Falias = |F - k Fs|, choosing integer k so the result lies in [0, Fs/2]
Quantization step size	Δ = Range / 2^b (b = number of bits)
Max quantization error (rounding)	±Δ/2

A.2 Discrete-Time Signals and Systems (Chapter 2)

Quantity	Formula
Convolution sum	y[n] = Σ x[k] h[n-k]
Convolution output length	Nx + Nh - 1
Causality (LTI)	h[n] = 0 for n < 0
Stability (LTI, BIBO)	Σ|h[n]| < infinity

A.3 The Z-Transform (Chapter 3)

Quantity	Formula
Z-transform	X(z) = Σ x[n] z^-n
Standard pair (causal)	a^n u[n] <-> 1/(1-a z^-1), ROC |z|>|a|
Standard pair (anticausal)	-a^n u[-n-1] <-> 1/(1-a z^-1), ROC |z|<|a|
Time shift	x[n-k] <-> z^-k X(z)
Convolution property	x1[n]*x2[n] <-> X1(z) X2(z)
Causal + stable condition	All poles strictly inside the unit circle

A.4 Frequency Analysis and DFT/FFT (Chapters 4-6)

Quantity	Formula
DTFT	X(e^jw) = Σ x[n] e^-jwn (periodic, period 2π)
N-point DFT	X[k] = Σ x[n] e^-j(2π/N)kn, n=0..N-1
Inverse DFT	x[n] = (1/N) Σ X[k] e^j(2π/N)kn
DFT bin to frequency	Fk = k Fs / N
Direct DFT complexity	O(N^2) multiplications
Radix-2 FFT complexity	O((N/2) log2 N) multiplications

A.5 Implementation and Filter Design (Chapters 7-8)

Quantity	Formula
FIR Direct Form resources (M+1 taps)	M+1 multipliers, M adders, M delay elements
Direct Form I delays	M + N
Direct Form II (canonical) delays	max(M, N)

Appendix B: MATLAB / Python (NumPy) Quick Reference

Most BSc DSP labs in Bangladesh use MATLAB or Python (NumPy/SciPy) to verify hand calculations and visualize signals. The table below maps the key operations of this manual to their corresponding commands.

Operation	MATLAB	Python (NumPy/SciPy)
Generate a sequence	n = 0:10; x = 0.5.^n;	n = np.arange(11); x = 0.5**n
Convolution	y = conv(x, h);	y = np.convolve(x, h)
Z-transform (symbolic, poles/zeros)	[z,p,k] = tf2zp(b,a);	from scipy.signal import tf2zpk; z,p,k = tf2zpk(b,a)
Frequency response	[H,w] = freqz(b,a);	from scipy.signal import freqz; w,H = freqz(b,a)
N-point DFT/FFT	X = fft(x, N);	X = np.fft.fft(x, N)
Inverse FFT	x = ifft(X);	x = np.fft.ifft(X)
Plot magnitude spectrum	plot(abs(X));	plt.plot(np.abs(X))
Pole-zero plot	zplane(b,a);	Use matplotlib with tf2zpk output, or `scipy.signal.zpk2tf`
FIR filter design (window method)	b = fir1(N, Wc);	from scipy.signal import firwin; b = firwin(N+1, Wc)
IIR filter design (Butterworth)	[b,a] = butter(N, Wc);	from scipy.signal import butter; b,a = butter(N, Wc)

Appendix C: Practice Examination (25 Questions)

This practice examination samples every chapter of this manual, with emphasis matching the manual's own emphasis on Chapters 1-4 and 7. An answer key follows.

42. State the Sampling Theorem and define Nyquist rate.
43. A signal has Fmax = 4 kHz. What is the minimum sampling rate to avoid aliasing?
44. Is x[n] = cos(0.4πn) periodic? If so, find its period.
45. Define what it means for a system to be BIBO stable.
46. A system has h[n] = (0.6)^n u[n]. Is it causal and stable?
47. State the convolution sum formula and the output-length rule.
48. Find the z-transform of x[n] = (0.4)^n u[n], with its ROC.
49. What is the causality condition on the ROC of a z-transform?
50. What is the stability condition on the poles of a causal system?
51. Find the inverse z-transform of X(z) = z/(z-0.3) (causal).
52. What is the DTFT, and how does it relate to the z-transform?
53. State the Convolution Theorem for the DTFT.
54. Name the four classic ideal filter types.
55. What condition is required for a stable causal inverse system to exist?
56. What does the DFT do that the DTFT cannot, from a computational standpoint?
57. Explain the difference between circular and linear convolution.
58. How many multiplications does a direct 64-point DFT require?
59. How many multiplications does a radix-2 FFT require for N=64?
60. What is required of N for the classic radix-2 FFT algorithm?
61. How many delay elements does Direct Form II need for M=N=4?
62. What is the difference between fixed-point and floating-point representation?
63. A 10-bit fixed-point system spans -1 to +1. Find the quantization step size.
64. Why can coefficient quantization make a designed-stable IIR filter actually unstable?

65. What is a limit-cycle oscillation?

66. Name one real-world DSP application and the chapter concept it relies on.

Answer Key

Q#	Answer
1	A signal band-limited to Fmax can be perfectly reconstructed if Fs >= 2Fmax; 2Fmax is the Nyquist rate
2	8 kHz
3	Yes; ω/2π = 0.2, rational (1/5), so periodic with N=5
4	Every bounded input produces a bounded output
5	Causal (h[n]=0 for n<0); stable (Σ|h[n]| = 1/(1-0.6) = 2.5, finite)
6	y[n]=Σx[k]h[n-k]; output length = Nx+Nh-1
7	X(z)=1/(1-0.4z^-1), ROC |z|>0.4
8	ROC extends outward to infinity: |z|>r
9	All poles strictly inside the unit circle
10	x[n] = (0.3)^n u[n]
11	DTFT is the z-transform evaluated on the unit circle (z=e^jw)
12	DTFT{x*h} = X(e^jw) H(e^jw) — convolution becomes multiplication
13	Lowpass, Highpass, Bandpass, Bandstop (Notch)
14	The system must be minimum-phase (all poles AND zeros inside the unit circle)
15	It samples the continuous DTFT at N points, making it computable/storable exactly on a digital machine
16	Circular convolution wraps around (period N); linear convolution does not — zero-padding to Nx+Nh-1 makes them equal
17	N^2 = 4096
18	(N/2)log2(N) = 32*6 = 192
19	N must be an exact power of 2
20	max(4,4) = 4
21	Fixed-point: integer with assumed binary point, fast/simple, limited range. Floating-point: mantissa x 2^exponent, wide dynamic range, more complex hardware
22	Δ = 2/2^10 = 2/1024 ≈ 0.00195
23	Rounding shifts actual pole locations; a high-order Direct Form filter is highly sensitive and a pole can move outside the unit circle
24	A persistent, small, unwanted oscillation in an IIR filter's output caused by accumulated round-off error, even with zero input
25	e.g., mobile voice calls rely on linear prediction (Chapters 2-3); OFDM in 4G/5G relies on the FFT (Chapter 6)

Glossary of Key Terms

Term	Definition
Aliasing	Distortion where high-frequency content masquerades as a lower frequency due to insufficient sampling rate
BIBO Stability	Bounded-Input, Bounded-Output stability — every bounded input produces a bounded output
Butterfly	The basic computational unit of the FFT, combining two inputs into two outputs with one multiplication and two additions
Causal System	A system whose output depends only on present and past inputs
Convolution	The operation combining an input with a system's impulse response to produce the output: y[n]=x[n]*h[n]
DFT	Discrete Fourier Transform — samples the DTFT at N equally spaced frequencies, computable on a digital machine
Direct Form I / II	Standard block-diagram realizations of an IIR difference equation; Direct Form II is canonical (minimum delay elements)
DTFT	Discrete-Time Fourier Transform — the z-transform evaluated on the unit circle; continuous and periodic in frequency
FFT	Fast Fourier Transform — an efficient O(N log N) algorithm for computing the DFT
Fixed-Point / Floating-Point	Two standard ways of representing numbers with finite precision in digital hardware
FIR / IIR	Finite / Infinite Impulse Response — nonrecursive (no feedback) vs. recursive (feedback) systems
Impulse Response	An LTI system's output when the input is a unit impulse δ[n]; fully characterizes the system
LTI System	Linear, Time-Invariant system — the central class of systems analyzable with convolution and the z-transform
Nyquist Rate	The minimum sampling rate (2×Fmax) required to avoid aliasing
Pole / Zero	Roots of the denominator/numerator of a system function H(z); determine stability and frequency shaping
Quantization	Rounding a continuous-amplitude value to the nearest representable discrete level
Region of Convergence (ROC)	The set of z values for which the z-transform sum converges
Sampling Theorem	States that Fs >= 2Fmax is required to perfectly reconstruct a band-limited signal from its samples
System Function H(z)	The z-transform of an LTI system's impulse response; equals Y(z)/X(z)
Z-Transform	A transform converting a sequence into a function of a complex variable z, generalizing the DTFT

Index

Aliasing....5
Anti-Aliasing Filter....5
BIBO Stability....13
Bilinear Transformation....45
Butterfly (FFT)....36
Butterworth Filter....45
Cascade Form....39
Causal System....13
Chebyshev Filter....45
Circular Convolution....32
Convolution Sum....13
Convolution Theorem (DTFT)....27
Correlation....13
DFT (Discrete Fourier Transform)....32
Difference Equation....13
Direct Form I....39
Direct Form II....39
DTFT....27
Elliptic Filter....45
Energy vs. Power Signal....13
FFT (Fast Fourier Transform)....36
FIR System....39
Fixed-Point Representation....39
Floating-Point Representation....39
Frequency Response....27
IIR System....39
Impulse Response....13
Linear Phase....45
Linear Time-Invariant (LTI) System....13
Magnitude Response....27
Minimum-Phase System....27
Nyquist Rate....5
Parallel Form....39
Parseval's Theorem....32
Phase Response....27
Pole-Zero Plot....21
Quantization....5
Quantization Error....39
Region of Convergence (ROC)....21
Sampling Theorem....5
State-Space Representation....39
System Function H(z)....21
Unit Impulse....13
Unit Step....13
Windowing Method (FIR design)....45
Z-Transform....21